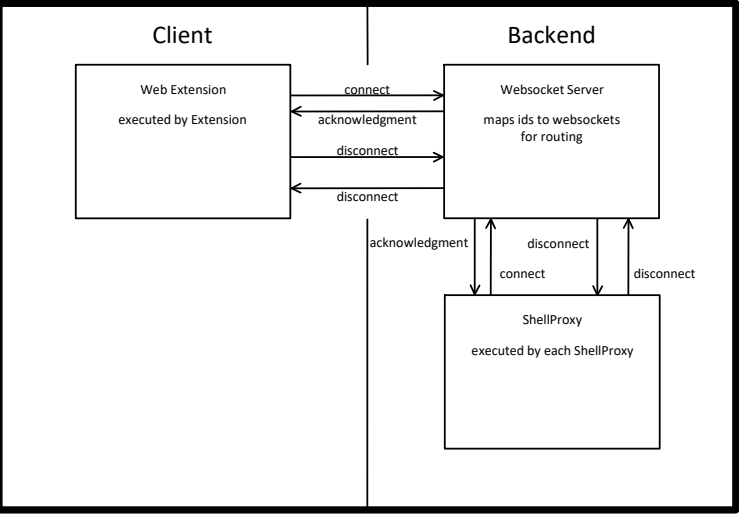


Connect & Disconnect



Connect message
send from Web Extension or ShellProxy to inform Websocket Server of new peer

- **when send from Web Extension to Websocket Server**
 - includes id=0 as unique identifier of Client
- **when send from ShellProxy to Websocket Server**
 - includes id=PID as unique identifier of ShellProxy

Websocket Server maps id to websocket and responds with acknowledgment

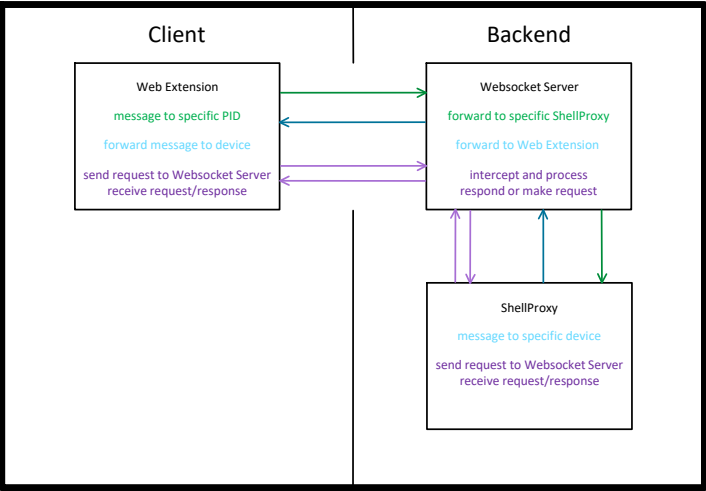
Disconnect message
can be send from anywhere to terminate a connection
is send from Websocket Server when receiving a non (dis-)connect message from an unknown id
no acknowledgment needed

Connect Message (*->WSS)		
index	content	description
0	"connect"	inform WSS of new connection
1	["client", 0] ["shell", id]	address of peer

Connect Acknowledgment (WSS->*)		
index	content	description
0	"connect ACK"	inform peer of connection success

Disconnect Message (*->*)		
index	content	description
0	"disconnect"	inform peer of connection termination

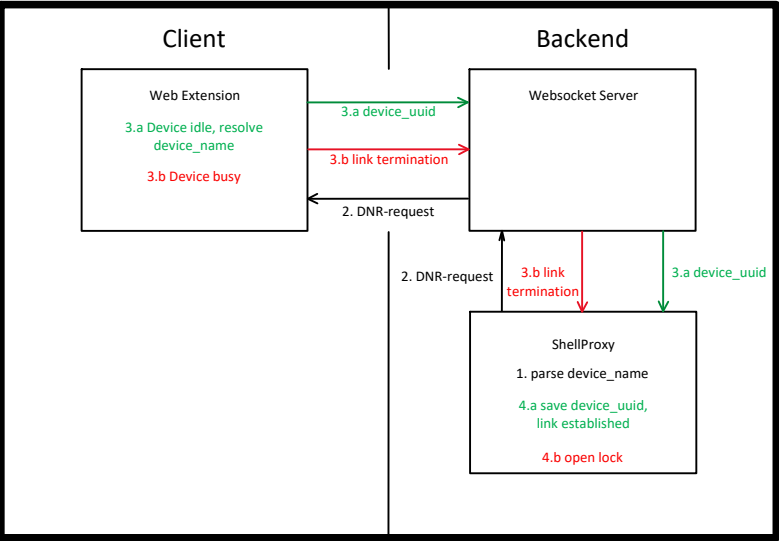
Websocket Server routing



After peers have connected, they are able to send messages
Websocket Server can forward or intercept messages, based on type (index 0)

- **Message from Client to ShellProxy**
 - Websocket Server checks type, decides to forward
 - from=["device", name], to=["shell", id], Websocket Server forwards message to ShellProxy with id
- **Message from ShellProxy to Device**
 - Websocket Server checks type, decides to forward
 - from=["shell", id], to=["device", name], Websocket Server forwards message to Client with id=0
 - (Client has to forward message according to device id)
- **Message intercepted by Websocket Server**
 - Websocket Server checks type, decides to intercept
 - is processed by Websocket Server
 - is able to respond or send own requests

Link Establishment from ShellProxy

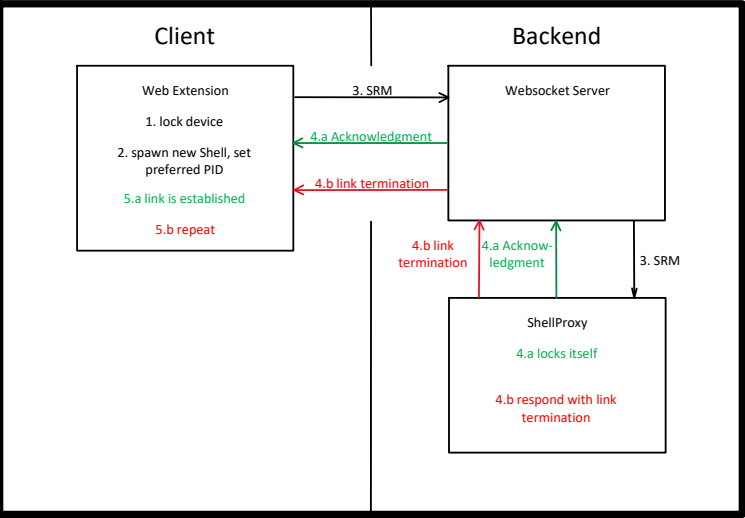


- Link establishment from ShellProxy**
executed when User accesses a Device from a Shell directly
1. ShellProxy intercepts command and parses device_name
 2. ShellProxy sends DNR-request (Device Name Resolution) to Client and locks itself
 3. Client responds to ShellProxy with one of the following
 - a. device_id when requested device is idle (locks device)
 - b. link termination when device is busy/locked
 4. ShellProxy performs one of the following
 - a. the link is established when receiving a device_id
 - b. opens lock when receiving link termination

Device Request Message		
index	content	description
0	"DRM"	inform device of new link
1	["shell", id]	address of sender
2	["device", name]	address of recipient

Device Request Message Acknowledgment		
index	content	description
0	"DRM ACK"	inform shell of successful link establishment
1	["device", name]	address of sender
2	["shell", id]	address of recipient

Link Establishment from Web Extension

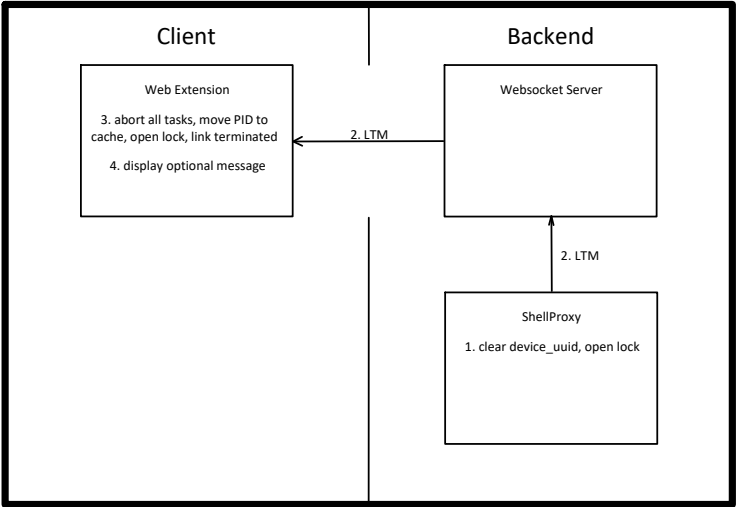


- Link establishment from Web Extension**
executed when User presses a Button in the Web Extension
Corresponding Device has either a cached/preferred PID or not
1. Device locks itself
 2. (if no cached PID present) spawn new Shell, set preferred PID to new Shell PID
 3. Device sends SRM (Shell Request Message) to preferred PID
 4. ShellProxy responds to Device
 - a. if ShellProxy is in non user mode and idle, respond with acknowledgment (locks shell)
 - b. if not, respond with link termination
 5. Device receives message
 - a. if acknowledgment, link is established
 - b. else repeat

Shell Request Message		
index	content	description
0	"SRM"	inform shell of new link
1	["device", name]	address of sender
2	["shell", id]	address of recipient

Shell Request Message Acknowledgment		
index	content	description
0	"SRM ACK"	inform device of successful link establishment
1	["shell", id]	address of sender
2	["device", name]	address of recipient

Link Termination from ShellProxy



Link termination from ShellProxy

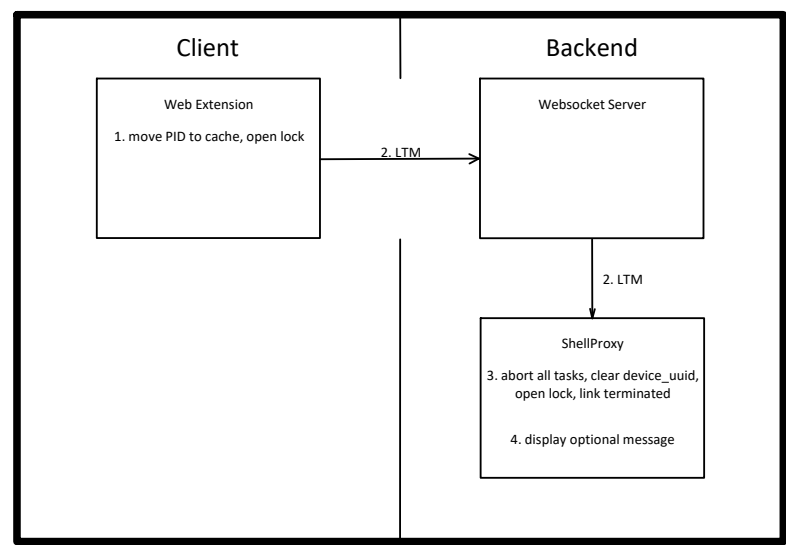
executed when:

- ShellProxy is linked and receives message from unknown device_uuid
- ShellProxy encounters fatal error
- User interrupts execution

1. ShellProxy clears device_uuid, opens lock
2. ShellProxy sends LTM (Link Termination Message), as (success or) error, with optional message
3. Device receives message, aborts all tasks, moves PID to cache, opens lock, link is terminated
4. optional message is displayed

Link Termination Message		
index	content	description
0	"LTM"	inform device of link termination
1	["shell", id]	address of sender
2	["device", name]	address of recipient
3	"success" "error"	class of termination
4	msg?	optional message

Link Termination from Web Extension



Link termination from Web Extension

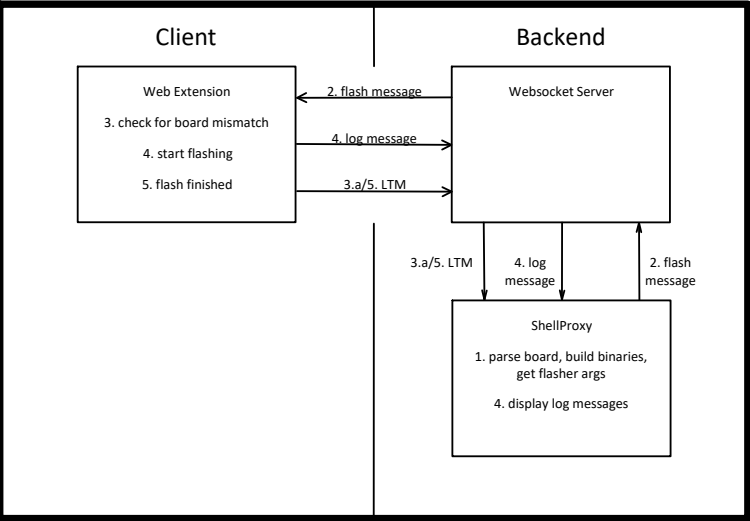
executed when:

- Device is linked and receives message from unknown PID
- Device encounters fatal error
- User interrupts execution
- Task execution is finished

1. move PID to cache, open lock
2. send LTM (Link Termination Message), as success or error, with optional message
3. ShellProxy receives message, aborts all tasks, clears device_uuid, opens lock, link is terminated
4. optional message is displayed

Link Termination Message		
index	content	description
0	"LTM"	inform shell of link termination
1	["device", name] ["client", 0]	address of sender
2	["shell", id]	address of recipient
3	"success" "error"	class of termination
4	msg?	optional message

Flash from ShellProxy



Flash from ShellProxy

Executed when:

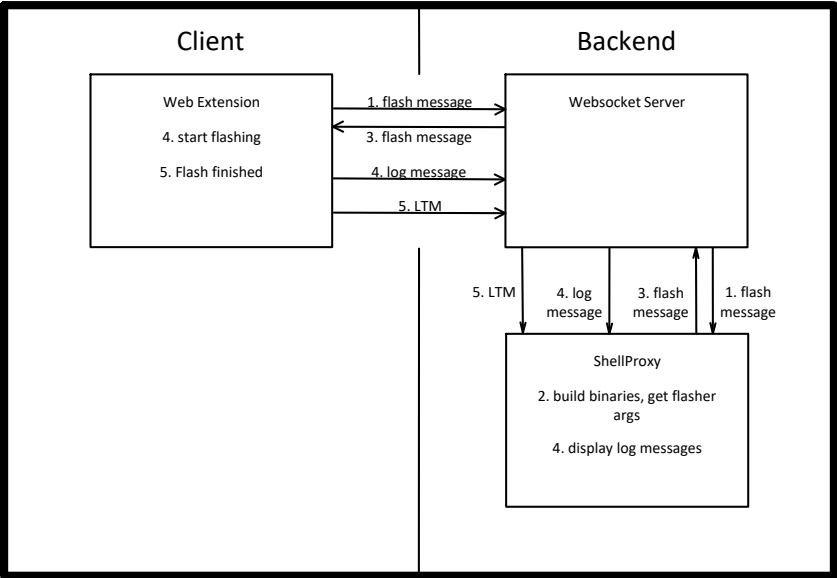
- User inputs make flash in Shell directly with port=device_name and board=device_board
- On error or interrupt anywhere at anytime:
- Error LTM is send

0. establish link from ShellProxy
1. parse board, build binaries, get flasher args
2. send flash message to device with board, binaries and flasher args
3. Device checks if correct board is used
 - a. sends error link termination if mismatched
4. device starts flashing, sends log messages to display in ShellProxy
5. On finish, device sends success link termination

Flash Message		
index	content	description
0	"flash"	inform device of flash
1	["shell", id]	address of sender
2	["device", name]	address of recipient
3	board	used board
4	binaries	binaries to flash (as map from address to binary)
5	arguments	arguments for flashing (as string)

Log Message		
index	content	description
0	"log"	inform shell of log message
1	["device", name]	address of sender
2	["shell", id]	address of recipient
3	"log" "error"	class of log message
4	msg	log message

Flash from Web Extension



Flash from Web Extension

Executed when:

- User presses flash button in Web Extension, has specified board

On error or interrupt anywhere at anytime:

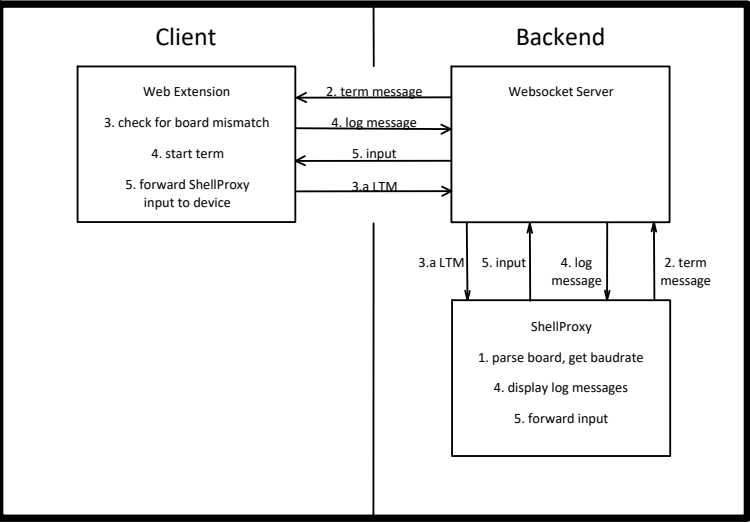
- Error LTM is send

0. establish link from Web Extension
1. send flash message to ShellProxy with board
2. build binaries, get flasher args according to board
3. send flash message to device with binaries, flasher args
4. device starts flashing, sends log messages to display in ShellProxy
5. On finish, device sends success link termination

Flash Request Message		
index	content	description
0	"flash request"	request shell to flash device
1	["device", name]	address of sender
2	["shell", id]	address of recipient
3	board	which board to use
4	projectPath	where to execute flash command

Flash Message		
index	content	description
0	"flash"	inform device of flash
1	["shell", id]	address of sender
2	["device", name]	address of recipient
3	binaries	binaries to flash (as map from address to binary)
4	arguments	arguments for flashing (as string)

Log Message		
index	content	description
0	"log"	inform shell of log message
1	["device", name]	address of sender
2	["shell", id]	address of recipient
3	"log" "error"	class of log message
4	msg	log message



Flash from ShellProxy

Executed when:

- User inputs make term in Shell directly with port=device_name and board=device_board

On error or interrupt anywhere at anytime:

- Error LTM is send

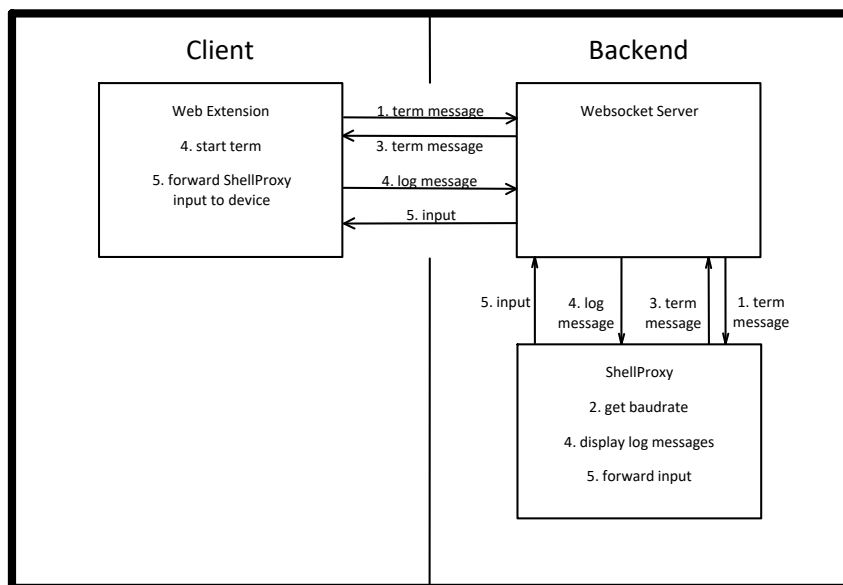
0. establish link from ShellProxy
1. parse board, get baudrate
2. send term message to device with board and baudrate
3. Device checks if correct board is used
 - a. sends error link termination if mismatched
4. device starts term, sends log messages to display in ShellProxy
5. ShellProxy forwards inputs to device

Term Message		
index	content	description
0	"term"	inform device of term
1	["shell", id]	address of sender
2	["device", name]	address of recipient
3	board	used board
4	baudrate	baudrate to use

Log Message		
index	content	description
0	"log"	inform shell of log message
1	["device", name]	address of sender
2	["shell", id]	address of recipient
3	"log" "error"	class of log message
4	msg	log message

Input Message		
index	content	description
0	"input"	inform device of input message
1	["shell", id]	address of sender
2	["device", name]	address of recipient
3	input	input message

Term from Web Extension



Flash from Web Extension

Executed when:

- User presses term button in Web Extension, has specified board

On error or interrupt anywhere at anytime:

- Error LTM is send

0. establish link from Web Extension

1. send term message to ShellProxy with board

2. get baudrate according to board

3. send term message to device baudrate

4. device starts term, sends log messages to display in ShellProxy

5. ShellProxy forwards inputs to device

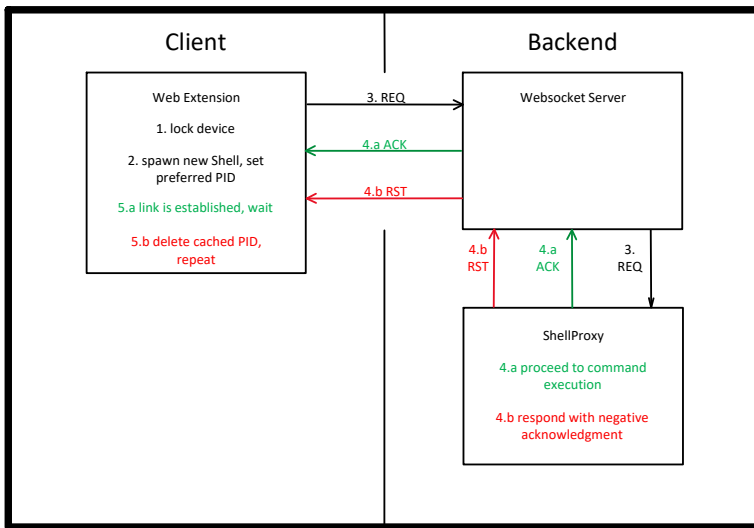
Term Request Message		
index	content	description
0	"term request"	request shell to flash device
1	["device", name]	address of sender
2	["shell", id]	address of recipient
3	board	which board to use
4	projectPath	where to execute term command (only necessary for access to riot library)

Input Message		
index	content	description
0	"input"	inform device of input message
1	["shell", id]	address of sender
2	["device", name]	address of recipient
3	input	input message

Log Message		
index	content	description
0	"log"	inform shell of log message
1	["device", name]	address of sender
2	["shell", id]	address of recipient
3	"log" "error"	class of log message
4	msg	log message

Term Message		
index	content	description
0	"term"	inform device of term
1	["shell", id]	address of sender
2	["device", name]	address of recipient
3	baudrate	baudrate to use

Command Request



Command Request

executed when:

- User presses a Button in the Web Extension

Device has either a cached PID or not

- Device locks itself
- (if no cached PID present) spawn new Shell, set cached PID to new shell PID
- Device sends RM (Request Message) to cached PID
- ShellProxy responds to Device
 - if ShellProxy is in non user mode and idle, respond with acknowledgment, proceed to command execution
 - if not, respond with termination
- Device receives message
 - if acknowledgment, link is established, wait for further actions
 - else delete cached PID, repeat from step 2

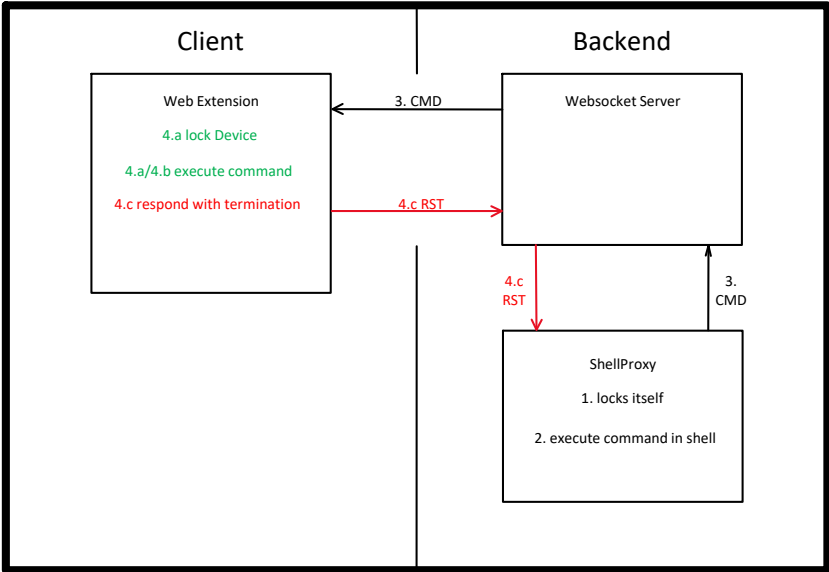
Shell Request Message		
index	content	description
0	"REQ"	inform shell of command
1	["device", name]	address of sender
2	["shell", id]	address of recipient
3	newInstance	true or false, depending on if extension spawned a new shell
4	command request	command that shell should prepare

Request Message Acknowledgment		
index	content	description
0	"ACK"	inform device of successful link establishment
1	["shell", id]	address of sender
2	["device", name]	address of recipient

Command Request		
index	content	description
0	"flash" "term"	type of command
1	board	which board to use
2	projectPath	where to execute command

Termination Message		
index	content	description
0	"RST"	inform device of unsuccessful link establishment
1	["shell", id]	address of sender
2	["device", name]	address of recipient
3	"success" "error"	class of termination
4	msg?	optional message

Command Execution



Command Execution

- executed when:
- User presses a Button in the Web Extension (after Request Message)
 - User enters command in shell
1. Shell locks itself
 2. Shell executes corresponding shell command
 3. Shell sends command to Device
- Device can have 3 states:
- a. Unlocked (user tries to access unoccupied device)
 - b. Locked to current Shell (after sending a request message)
 - c. Locked to a different Shell (user tries to access occupied device)
4. Device takes action based on state
 - a. locks itself and executes command
 - b. execute command
 - c. respond with termination

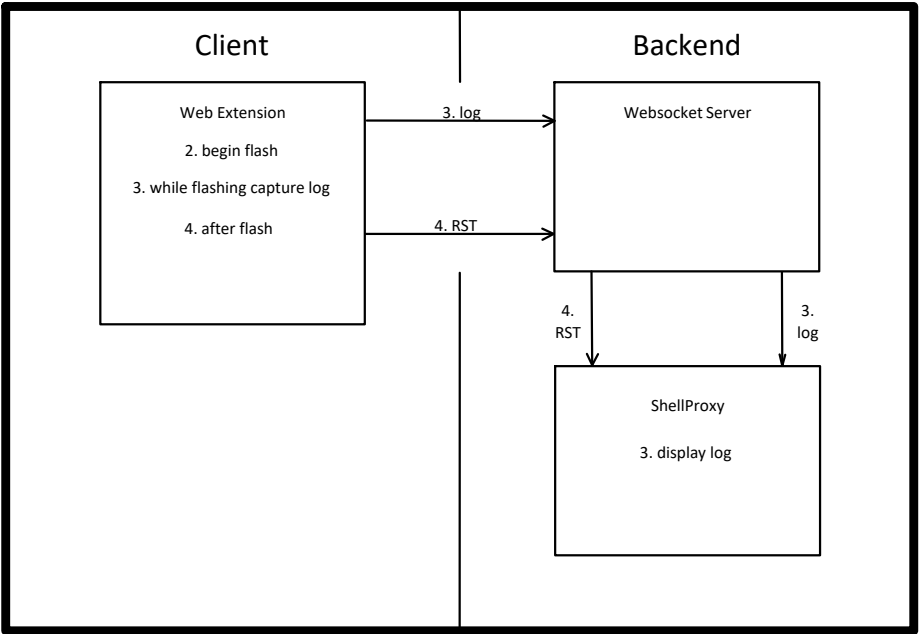
Command Execution Message		
index	content	description
0	"CMD"	execute command on device
1	["shell", id]	address of sender
2	["device", name]	address of recipient
3	flashCommand termCommand	command that device should execute

Termination Message		
index	content	description
0	"RST"	inform device of unsuccessful link establishment
1	["shell", id]	address of sender
2	["device", name]	address of recipient
3	"success" "error"	class of termination
4	msg?	optional message

Flash Command		
index	content	description
0	"flash"	type of command
1	board	used board
2	binaries	binaries to flash (as map from address to binary)
3	arguments	arguments for flashing (as string)

Term Command		
index	content	description
0	"term"	type of command
1	board	used board
2	baudrate	baudrate to use

Flash Command

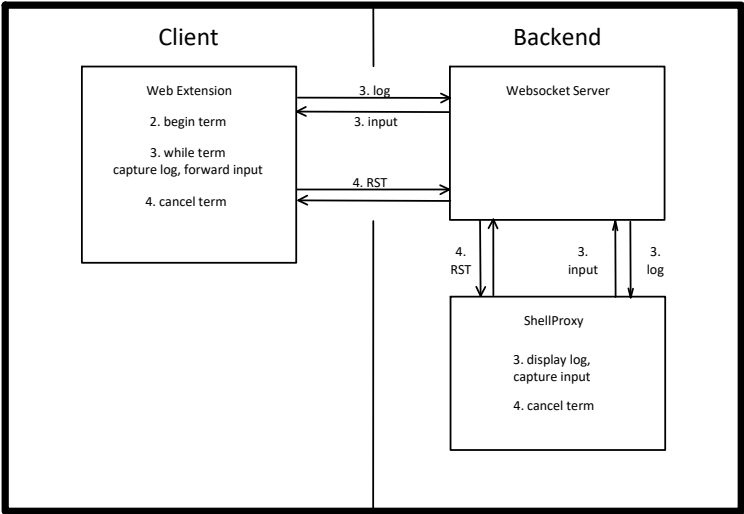


- Flash Command**
- 1. device receives flash command
 - 2. device starts flashing
 - 3. device sends log messages to display in ShellProxy
 - 4. On finish, device sends success termination

Log Message		
index	content	description
0	"log"	inform shell of log message
1	["device", name]	address of sender
2	["shell", id]	address of recipient
3	"log" "error"	class of log message
4	msg	log message

Termination Message		
index	content	description
0	"RST"	inform device of successful link termination
1	["shell", id]	address of sender
2	["device", name]	address of recipient
3	"success"	class of termination
4	msg?	optional message

Term Command



- Term Command**
- 1. device receives term command
 - 2. device starts term
 - 3. device sends log messages to display in ShellProxy, ShellProxy sends input to send to device
 - 4. On cancel (from either side), send link termination

Log Message		
index	content	description
0	"log"	inform shell of log message
1	["device", name]	address of sender
2	["shell", id]	address of recipient
3	"log" "error"	class of log message
4	msg	log message

Termination Message		
index	content	description
0	"RST"	inform device of link termination
1	["shell", id]	address of sender
2	["device", name]	address of recipient
3	"success" "error"	class of termination
4	msg?	optional message

Input Message		
index	content	description
0	"input"	forward message to device
1	["device", name]	address of sender
2	["shell", id]	address of recipient
3	msg	input that is forwarded to device